SECTION B - DEBUGGING CHALLENGES

15 Questions · Each presents a real code snippet - identify what happens and why

These mirror the "what does this print / what is the bug" format used in real technical interviews.

Q127. [DEBUG] *(Medium)*

```
a = np.array([1, 2, 3])
b = a
b[0] = 99
print(a[0])
```

What does this print, and why?

a) 1 - b is independent of a b) 99 - `b = a` creates an alias, not a copy; both names reference the same underlying array c) Raises an error d) 0

Q128. [DEBUG] *(High)*

```
prices = np.array([10, 20, 30, 40])
discounted = prices[1:3]
discounted *= 0.5
print(prices)
```

What happens when this code runs?

a) prices becomes [10,10,15,40] b) Raises a casting error (TypeError) - in-place `*=` on an int64 array cannot safely cast the float64 result back into int64 under NumPy's "same_kind" casting rule c) discounted becomes [10,15], prices unaffected d) Silently truncates to [10,10,15,40] with no error

Q129. [DEBUG] *(High)*

```
def process(arr, multiplier=[]):
    multiplier.append(2)
    return arr * len(multiplier)

print(process(np.array([1,2,3])))
print(process(np.array([1,2,3])))
```

What is printed by the two calls?

a) [1,2,3] then [1,2,3] - multiplier resets each call b) [1,2,3] then [2,4,6] - the default mutable list is evaluated ONCE at function definition and persists across calls, so it keeps growing c) Raises an error on the second call d) [2,4,6] then [4,8,12]

Q130. [DEBUG] *(Medium)*

```
a = np.array([1, 2, 3, 4, 5])
mask = a > 3
```

```
a = a[mask]
mask = a > 10
print(a[mask])
```

What prints?

a) [4,5] b) [] - an empty array, since no remaining elements exceed 10 c) Raises an IndexError d) [1,2,3,4,5]

Q131. [DEBUG] *(Medium)*

```
m = np.array([[1,2],[3,4]])
total = 0
for row in m:
    for val in row:
        total += val
avg = total / m.size
print(avg)
```

Does this code correctly compute the matrix mean? What is the more idiomatic NumPy equivalent?

a) No, it is wrong; the correct value is m.sum() b) Yes, it correctly computes the mean (2.5), but the idiomatic, far faster equivalent is simply `m.mean()` - avoid nested Python loops over NumPy arrays c) No, it raises a TypeError d) Yes, but only because m happens to be square

Q132. [DEBUG] *(Medium)*

```
scores = np.array([55, 70, 85, 90, 40])
passed = scores[scores >= 60]
passed[0] = 100
print(scores)
```

What does `scores` print after this code runs?

a) [55,70,85,90,40] - unaffected, because boolean-mask indexing on the right-hand side returns a copy b) [55,100,85,90,40] c) Raises an error d) [100,70,85,90,40]

Q133. [DEBUG] *(Medium)*

```
data = np.array([10, 20, 30, 40, 50])
indices = [4, 2, 0]
result = data[indices]
print(result)
```

What does this print, and what indexing concept does it demonstrate?

a) [10,20,30] - slicing b) [50,30,10] - fancy indexing returns elements in the EXACT order the index array specifies, not the original array's order c) Raises an IndexError d) [10,30,50]

Q134. [DEBUG] *(Medium)*

```
arr = np.zeros(5)
for i in range(5):
    arr[i] = i ** 2
print(arr.dtype)
```

What is `arr.dtype` after this loop, and why might that be a subtle bug?

a) int64 - assigning integers changes the dtype automatically b) float64 - `np.zeros(5)` creates a float64 array, and assigning integer values into it SILENTLY CASTS them to float; the array never becomes an integer array c) object d) Raises a TypeError on assignment

Q135. [DEBUG] *(High)*

```
def normalize(arr):
    return (arr - arr.min()) / (arr.max() - arr.min())

data = np.array([5.0, 5.0, 5.0])
print(normalize(data))
```

What happens when this runs?

a) Returns [0,0,0] cleanly b) Produces [nan, nan, nan] with a RuntimeWarning - dividing by zero (max-min=0) when all values are identical; needs a guard clause for constant arrays c) Raises a fatal ZeroDivisionError, crashing the program d) Returns [1,1,1]

Q136. [DEBUG] *(Medium)*

```
a = np.array([1, 2, 3])
b = np.array([4, 5])
print(a + b)
```

What happens?

a) [5,7,3] b) Raises ValueError - shapes (3,) and (2,) are not broadcast-compatible (neither matches, neither is 1) c) [5,7] d) Pads b with a 0 automatically

Q137. [DEBUG] *(Medium)*

```
matrix = np.arange(20).reshape(4,5)
col = matrix[:, 2]
col[0] = -1
print(matrix[0, 2])
```

What prints?

a) 2 - unaffected b) -1 - column slicing returns a view sharing memory with the original matrix c) Raises an error d) 0

Q138. [DEBUG] *(High)*

```
def get_outliers(data, threshold=2):
    z = (data - data.mean()) / data.std()
```

```
    return data[abs(z) > threshold]

result = get_outliers(np.array([1.0, 1, 1, 1, 1]))
print(result)
```

What is the likely problem with this function as written, given the input shown?

a) No problem, it always works perfectly b) When `data.std()` is 0 (a constant array), division by zero produces NaN z-scores; comparisons against NaN are always False, so the function silently returns an empty array instead of raising a clear error or handling the case explicitly c) `abs()` does not work on NumPy arrays d) threshold should always be 3, never 2

Q139. [DEBUG] *(Medium)*

```
a = np.array([3, 1, 4, 1, 5, 9, 2, 6])
sorted_unique = np.unique(a)
print(sorted_unique[-1])
```

What does this print, and why is it a clean way to find the maximum?

a) 9 - np.unique() returns values SORTED ascending by default, so [-1] is the largest unique value
b) 3 - the first element of the original array c) Raises an error d) [9] as a single-element array

Q140. [DEBUG] *(High)*

```
def safe_log(arr):
    return np.log(arr)

prices = np.array([100, 0, 50, -10])
log_prices = safe_log(prices.astype(float))
print(np.isnan(log_prices).sum())
```

What does this print?

a) 0 - no NaNs produced b) 1 - log(-10) returns NaN (log of a negative number is undefined); log(0) returns -inf, which is NOT counted as NaN, so only 1 NaN total c) 2 d) Raises a fatal error, crashing before reaching the print statement

Q141. [DEBUG] *(High)*

```
def batch_normalize(batches):
    results = []
    for b in batches:
        results.append((b - b.mean()) / b.std())
    return np.array(results)

a = np.array([1,2,3])
b = np.array([1,2,3,4])
batch_normalize([a, b])
```

What happens when this runs?

a) Returns a clean (2, N) array b) Raises a ValueError when converting `results` (a Python list containing differently-shaped arrays) into `np.array(results)` - NumPy cannot stack a length-3 and a length-4 array into one rectangular array without explicitly passing dtype=object c) Silently pads the shorter array with zeros d) Works fine, returning shape (2,4)

			indexed result never affects the source array.
124	**B**		Single-argument np.where() is equivalent to np.nonzero() - it locates positions, not values; for a 1D array, unpack with `[0]`.
125	**B**		NumPy's convention for argmax/argmin with ties is deterministic - always the first matching position.
126	**B**		This is one of the most frequent real-world argsort mistakes - always remember argsort defaults to ascending order.
127	**B**	**DEBUG**	Plain assignment (`b = a`) never copies array data - it just creates a second name pointing at the same object.
128	**B**	**DEBUG**	Verified directly: this raises "Cannot cast ufunc multiply output from dtype float64 to dtype int64 with casting rule same_kind". A common gotcha when slicing-then-modifying an int array with a float operation - fix by using a float dtype array, or `discounted[:] = discounted * 0.5` with explicit casting.
129	**B**	**DEBUG**	Verified directly. This is the classic "mutable default argument" trap in

			Python - it applies to any NumPy preprocessing function that uses a mutable default like `[]` or `{}`.
130	**B**	**DEBUG**	After the first filter, a=[4,5]. The second mask (a>10) matches nothing, so indexing with it returns an empty array - not an error.
131	**B**	**DEBUG**	Manually looping defeats the entire purpose of using NumPy - always reach for the built-in vectorized method first.
132	**A**	**DEBUG**	`passed` is an independent copy; modifying it never propagates back to the original `scores` array.
133	**B**	**DEBUG**	Fancy indexing follows the order of the supplied index array - useful for both reordering and selecting in one step.
134	**B**	**DEBUG**	A new array's dtype is fixed at creation time; element-wise assignment casts incoming values to match it, never the other way around.
135	**B**	**DEBUG**	Verified directly. A robust normalize() should check `if arr.max() == arr.min()` and handle the constant-array case explicitly before dividing.

136	**B**	**DEBUG**	Broadcasting has strict rules; arrays of genuinely different, incompatible lengths simply cannot combine.
137	**B**	**DEBUG**	Basic slicing always returns a view, regardless of whether you slice rows, columns, or both.
138	**B**	**DEBUG**	Verified: this returns an empty array (no exception, no warning surfaced to the caller) - a silent failure mode that's worse than a crash because it can hide a real problem from downstream code.
139	**A**	**DEBUG**	Although `a.max()` is the more direct and efficient way to get the maximum, this demonstrates that unique() guarantees a sorted result.
140	**B**	**DEBUG**	Verified directly: -inf and nan are distinct floating-point special values; isnan() only flags the genuinely undefined nan, not -inf.
141	**B**	**DEBUG**	Verified directly: "ValueError: setting an array element with a sequence. The requested array has an inhomogeneous shape." A frequent bug when batching variable-length results - keep them as a Python list of arrays instead of forcing a